Method of identifying urban heat islands by remote sensing based on big data

Minh Tuan Le[1,*], *Natalia* Bakaeva [1], *Nina* Danilina[1] and *Thi Van Anh* Hoang[2]

[1] Moscow State University of Civil Engineering, Moscow, Russian Federation
[2] Moscow Aviation Institute, Moscow, Russian Federation

Abstract. In recent decades, the speed of construction, expansion, and development of smart cities by humans has been increasing faster, thanks to new technologies. However, alongside the fantastic development of new technologies also comes negative impacts such as climate change. One of the characteristics of climate change is the changing thermal comfort of humans living in the city. Heat stress is a feature of the urban heat island phenomenon. Congestion of airflows within the urban area causes localized hot areas, causing a temperature difference of 2-3 degrees between the areas inside the city and the suburbs. This study presents a method for determining surface temperature and the location of heat islands based on Google's cloud computing platform. The platform uses a combination of satellite imagery, Earth observation data, and machine learning algorithms to allow users to identify and measure changes in land use, ecosystems, and climate patterns at a global scale. Google Earth Engine enables researchers and organizations to process enormous amounts of data with short turnaround times. The method described in the article allows the analysis and retrieval of data in the large dataset of the Landsat 8 satellite. By determining the location of heat islands, the government, policymakers, and planners can develop plans to cope with the urban heat island phenomenon and improve the quality of life for residents.

1 Introduction

In recent years, human activities, mainly caused by anthropogenic factors, have resulted in unprecedented climate change and environmental degradation, making global warming and energy shortage extremely apparent. As a result, there has been a significant shift in people's attitudes and behaviors, urging them to take strict measures to reduce carbon emissions. Specifically, in the building sector, which is a significant contributor to energy consumption and greenhouse gas emission, several actions and best practices have been implemented to promote energy efficiency and reduce carbon emissions.

Despite international efforts to limit global warming to 1.5 °C, city temperatures have surpassed those of surrounding rural or suburban areas by 2-5 °C, and in extreme cases, even up to 12 °C. This phenomenon, known as the urban heat island (UHI), is often exacerbated by heat waves - a common manifestation of climate change - leading to

* Corresponding author: architect290587@gmail.com

sustained and severe urban warming [1]. As a result, outdoor thermal comfort is greatly compromised, and indoor environments are also significantly affected [2].

The United States is the leader in implementing and monitoring urban surface temperatures on a large scale with the support of Landsat-1 head-to-head remote sensing technology. Eight generations of Landsat satellites have successfully launched into space, but only 7 and 8 satellites are active [3].

For over 40 years, the optical remote sensing satellite Landsat has consistently provided unprecedented amounts of earth observation data, totaling over 8 million images, the longest continuous record of medium spatial resolution data available [4]. Landsat data holds tremendous potential for providing high-quality, long-term, and large-area data records [5], [6]. User-friendly composited mosaics generated from Landsat data can support assessments of environmental and land-cover changes over decades, production of reflectance-based biophysical products, and fusion of reflectance data from various sensors [7]–[9]. This has made it possible to create models for deforestation over multiple years, estimate water quality [10]–[13], and study land surface phenology variability on national, continental, or even global scales, typically exceeding 1 million square kilometers in the area [14].

Regrettably, harnessing the full potential of these resources still presents significant challenges that require considerable technical skills and exertion. One of the most prominent obstacles is the management of essential information technology (IT) components, such as data acquisition and storage, processing of complex file formats, databases administration, allocation and management of machines, jobs, and job queues, utilization of CPU, GPU, and networking resources, as well as utilizing a wide array of geospatial data processing frameworks.

The cloud-based platform, Google Earth Engine, allows users to easily access high-performance computing resources for processing massive geospatial datasets, removing hurdles associated with IT constraints. In contrast to typical supercomputing facilities, Earth Engine is designed to aid researchers in sharing their findings with peers, NGOs, field workers, policymakers, and the general audience. Upon developing algorithms on Earth Engine, users can create structured data products or even interactive applications, taking advantage of the resources provided by Earth Engine, all without requiring specialized knowledge in application development, web programming, or HTML.

2 Materials and Methods

2.1 Platform Overview

The Google Earth Engine relies on multiple technologies within the Google data center, such as Flume Java framework, Bigtable, Spanner distributed databases [15], [16], Borg cluster management system, Colossus, and Google Fusion Tables [17], [18]. GEE provides accessibility via its API and web-based interface, enabling swift prototyping and visualization of outcomes. The GEE Data Catalogue presents a wide assortment of geospatial data, comprising satellite images that are pre-processed, optimized, and available in a format that mitigates data management issues, leading to amplified computing efficiency. Additionally, the data is accessible to the masses.

Google Earth Engine (GEE) offers users flexibility and customization through its data structure based on 2D raster bands stored in an "image" container. This structure allows for images that do not conform to standard data types, projections, or band counts. GEE's API

offers various operators that facilitate efficient parallel processing and enable users to transform their data effectively. Users can access the API through libraries or a web-based interactive environment, and the GEE user interface can be accessed via the Earth Engine homepage. The user guide is designed to be easily understandable by beginners, so specialized GIS, remote sensing, or scripting expertise is not required.

2.2 The database

This article uses surface reflectance (SR) products obtained from Landsat 8 OLI sensors (LANDSAT/LC08/C01/T1_SR) to produce NDVI composites. Data is valid from April 11, 2013, to the present. As a result of the near-polar orbits of Landsat satellites, aerial observation of each region occurs twice every 16 days, with two satellites working synchronously, leading to an eight-day revisit time for a given area.

2.3 Study Area

Moscow is the capital and largest city in Russia, located in the country's western region along the banks of the Moskva River. With a population of over 12 million people, it is one of the most populous cities in Europe. Moscow is considered Russia's political, cultural, and economic center, with a rich history dating back to the 12th century. The area of Moscow is about 2561 km2 (2020) (figure 1)

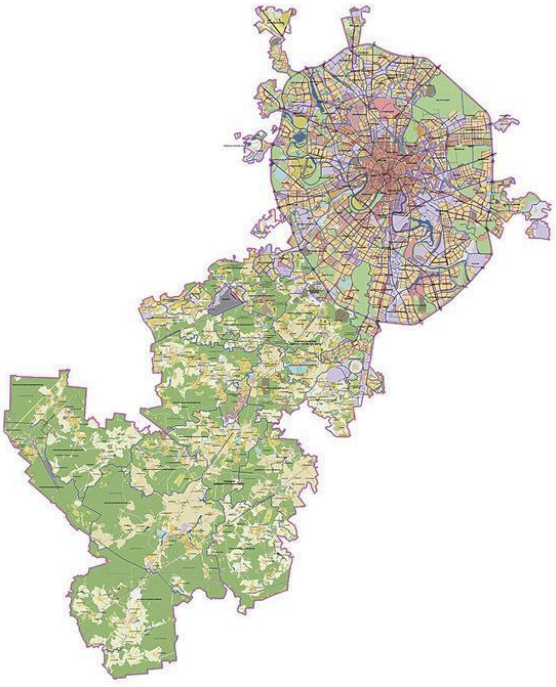

Fig. 1. The administrative division layout of the city of Moscow

2.4 Methodology

To perform the calculation of surface temperature through images provided by satellites under the support of the GEE platform, it is necessary to perform the following steps:

Step 1: Study area inspection.

The Moscow boundary map is provided through the GIS, in .shp format and loaded into the GEE platform with the following command:

```
geometry = ee.FeatureCollection("projects/ee-architect290587/assets/RU-MOW-20210318/boundary-polygon");
// Change the filter to your home city or any urban area of your choice
//var urban = ee.FeatureCollection("projects/ee-architect290587/assets/RU-MOW-20210318/boundary-polygon")
Map.addLayer(geometry,{}, 'geometry')
// Change the filter to your home city or any urban area of your choice
var filtered = geometry.filter(ee.Filter.eq('system:index', '0000000000000000003'))
var geometry = filtered.geometry()
Map.centerObject(geometry, 8);
var rgbVis = {
  min: 0.0,
  max: 3000,
  bands: ['B4', 'B3', 'B2'],
};
```

Step 2: Select the sensor, define the period, and cloud mask

The dataset used in the study was retrieved from the data generated by the Landsat 8 OLI/TIRS sensor. This dataset contains atmospherically corrected surface reflectance and land surface temperature derived. To perform Landsat 8 data retrieval and study time setting, use the following command line:

```
var L8 = ee.ImageCollection('LANDSAT/LC08/C01/T1_SR')
var filtered = L8.filterMetadata('CLOUD_COVER', 'less_than', 10)
.filterDate('2021-01-01','2021-12-31')
.filter(ee.Filter.bounds(geometry))
  print(filtered.toList(filtered.size()));
```

For the Landsat satellite on the GEE platform, the library has built-in algorithms to handle pixels covered by clouds. Specifically, in the case of Landsat 8 satellite using the "maskL8sr" algorithm, use the following command line:

```
//cloud mask
function maskL8sr(image) {
  // Bits 3 and 5 are cloud shadow and cloud, respectively.
  var cloudShadowBitMask = (1 << 3);
  var cloudsBitMask = (1 << 5);
  // Get the pixel QA band.
  var qa = image.select('pixel_qa');
  // Both flags should be set to zero, indicating clear conditions.
  var mask = qa.bitwiseAnd(cloudShadowBitMask).eq(0)
                 .and(qa.bitwiseAnd(cloudsBitMask).eq(0));
  return image.updateMask(mask);
}
```

Step 3: Compositing paradigm

Composite image visualization from bands (4, 5, 6) and bands (2, 3, 4) are used as input data for land surface temperature calculation. Use the following command line:

```
//vis params
var vizParams = {
bands: ['B5', 'B6', 'B4'],
min: 0,
max: 4000,
gamma: [1, 0.9, 1.1]
};
var vizParams2 = {
bands: ['B4', 'B3', 'B2'],
min: 0,
max: 3000,
gamma: 1.4,
};
```

Step 4: Mosaicking

Based on the NDVI vegetation index, the surface emissivity can be calculated by the method proposed by Valor E. and Caselles V. (1996). The emissivity of a pixel is calculated as the sum of the emissivity of its components according to the formula:

$$\varepsilon = \varepsilon_v P_v + \varepsilon_s (1 - P_v) \quad (1)$$

In which: ε_v; ε_s - emissivity characteristic for homogeneous soil and vegetation; P_v - the ratio of vegetation in a pixel. P_v has a value of 0 for bare soil and 1 for an area covered with vegetation.

$$P_v = \left[\frac{NDVI - NDVI_{min}}{NDVI_{max} - NDVI_{min}}\right]^2 \quad (2)$$

NDVI is determined according to formula (3), with NIR and RED being the value of electromagnetic wave reflection at near-infrared and red channels.

$$NDVI = \frac{NIR - RED}{NIR + RED} \quad (3)$$

The index NDVI of the study area is calculated in GEE using the following command lines:

```
{
var ndvi = image.normalizedDifference(['B5', 'B4']).rename('NDVI');
var ndviParams = {min: -1, max: 1, palette: ['blue', 'white',
'green']};
print(ndvi,'ndvi');
Map.addLayer(ndvi.clip(geometry), ndviParams, 'ndvi');
}
```

The minimum and maximum values of the NDVI index are calculated according to the following command lines:

```
// find the min and max of NDVI
{
var min = ee.Number(ndvi.clip(geometry).reduceRegion({
reducer: ee.Reducer.min(),

scale: 30,
maxPixels: 1e9
}).values().get(0));
print(min, 'min');

var max = ee.Number(ndvi.clip(geometry).reduceRegion({
reducer: ee.Reducer.max(),

scale: 30,
maxPixels: 1e9
}).values().get(0));
print(max, 'max')
}
```

The spectral irradiance value of the thermal infrared image is used to calculate the temperature. This temperature is also known as radiant temperature or brightness temperature. For Landsat Thermal infrared image data, the brightness temperature is determined by the formula:

$$T_b = \frac{K_2}{\ln ln\left(1+\frac{K_1}{L_\lambda}\right)} \quad (4)$$

Where: T_b Is the radiant temperature value of the image ($°K$);

L_λ - is the spectral irradiance value;

K_1, K_2 - constants provided from Landsat image data metadata file.

After determining the surface emissivity, the brightness temperature will be corrected to obtain the surface temperature value. The land surface temperature (LST) is determined as follows:

$$LST = \frac{T_b}{1+\left(\frac{\lambda T_b}{\rho}\right)*\ln ln\,\varepsilon} \quad (5)$$

Where: T_b is the radiant temperature value of the image ($°K$);

λ - is the value of the central wavelength of the thermal infrared range for channels 10 and 11 of Landsat 8 images.

ε - is the surface emissivity;

ρ - is the Stefan-Boltzmann constant ($1.38.10^{-23}$J/K);

The surface temperature and its maximum and minimum values are calculated in the google earth engine (GEE) based on the following commands:

```
//Emissivity
var a= ee.Number(0.004);
var b= ee.Number(0.986);
var EM=fv.multiply(a).add(b).rename('EMM');
var imageVisParam3 = {min: 0.9865619146722164, max:0.989699971371314};
Map.addLayer(EM, imageVisParam3,'EMM');

//LST in Celsius Degree bring -273.15
//NB: In Kelvin don't bring -273.15
var LST = thermal.expression(
'(Tb/(1 + (0.00115* (Tb / 1.438))*log(Ep)))-273.15', {
'Tb': thermal.select('B10'),
'Ep': EM.select('EMM')
}).rename('LST');

//min max average LST
{
var min = ee.Number(LST.reduceRegion({
reducer: ee.Reducer.min(),
scale: 30,
maxPixels: 1e9
}).values().get(0));
print(min, 'minLST');

var max = ee.Number(LST.reduceRegion({
reducer: ee.Reducer.max(),
scale: 30,
maxPixels: 1e9
}).values().get(0));
print(max, 'maxLST')
```

Step 5: Mosaic strategy and define urban heat island

In urban regions, the UHI effect is crucial in impacting the environment and the well-being of urban dwellers. Various thermal comfort indices evaluate UHI's effect on urban life quality. This study utilized the Urban Thermal Field Variance Index (UTFVI), calculated through equation (6). The study used the Urban Thermal Field Variance Index (UTFVI) To determine the location of UHIs in Moscow.

$$UTFVI = \frac{T_s - T_m}{T_s} \tag{6}$$

T_s is the ground surface temperature, and T_m is the average ground surface temperature.

The Urban Thermal Field Variance Index (UTFVI) values were categorized into six groups corresponding to an environmental assessment interpretation. Table 1 presents the threshold values for each urban area's six thermal field dispersion index categories.

Table 1. The threshold values of the Urban Thermal Field Variance Index (UTFVI) and the ecological evaluation index [21]

UTFVI (Urban Thermal Field Variance Index)	Urban heat island (UHI)	Ecological evaluation index
< 0.000	None	Excellent
0.000 – 0.005	Weak	Good
0.005 – 0.010	Middle	Normal
0.010 – 0.015	Strong	Bad
0.015 – 0.020	Stronger	Worse
➢ 0.020	Strongest	Worst

The surface temperature image of the study area and the location of the heat islands are calculated according to the following command lines:

```
Map.addLayer(LST, {min: 13.93, max:43.43, palette: [
'040274', '040281', '0502a3', '0502b8', '0502ce', '0502e6',
'0602ff', '235cb1', '307ef3', '269db1', '30c8e2', '32d3ef',
'3be285', '3ff38f', '86e26f', '3ae237', 'b5e22e', 'd6e21f',
'fff705', 'ffd611', 'ffb613', 'ff8b13', 'ff6e08', 'ff500d',
'ff0000', 'de0101', 'c21301', 'a71001', '911003'
]},'LST');

Export.image.toDrive({
  image: LST,
  description: 'LST_2021',
  folder: "image EE",
  scale: 30,
  region: geometry,
  fileFormat: 'GeoTIFF',
  formatOptions: {
    cloudOptimized: true
  }
});
//calculate index UTFVI

{
var UTFVI =thermal.expression(
'((Ts-24.811209695879064)/Ts)', {
'Ts': LST.select('LST'),

}).rename('UTFVI');
Map.addLayer(UTFVI, {min: -0.0001, max:0.02, palette: [
'040274', '040281', '0502a3', '0502b8', '0502ce', '0502e6',
'0602ff', '235cb1', '307ef3', '269db1', '30c8e2', '32d3ef',
'3be285', '3ff38f', '86e26f', '3ae237', 'b5e22e', 'd6e21f',
'fff705', 'ffd611', 'ffb613', 'ff8b13', 'ff6e08', 'ff500d',
'ff0000', 'de0101', 'c21301', 'a71001', '911003'
]},'UTFVI');

Export.image.toDrive({
  image:UTFVI,
  description: 'UTFVI_2016',
  folder: "image EE",
  scale: 30,
  region: geometry,
  fileFormat: 'GeoTIFF',
  formatOptions: {
    cloudOptimized: true
  }
})
}
```

3 Results

When applying the time filter from January 1, 2021, to December 31, 2021, with a cloud cover of about 10%, 12 images meet the criteria from Landsat 8 (Table 2).

Table 2. The data is provided by the Landsat 8 satellite.

Id image	Cloud cover (%)	Earth-sun distance (Astronomical Units)	Solar azimuth angle (°)
LC08_177021_20210410	2.38	1.001916	160.121277
LC08_178021_20210604	0.01	1.01452	156.582214
LC08_178021_20210620	0.24	1.016183	154.926315
LC08_178021_20210706	0.2	1.016729	154.094742
LC08_178022_20210620	1.75	1.016183	152.999771
LC08_178022_20210706	7.83	1.016729	152.194443
LC08_178022_20211010	0.45	0.998599	166.556564
LC08_179021_20210219	7.39	0.988583	160.899368
LC08_179021_20210510	0	1.009831	159.012573
LC08_179021_20210713	3.67	1.016566	154.125305
LC08_179022_20210510	0	1.009831	157.436035
LC08_179022_20210713	4.65	1.016566	152.272507

Urban heat islands (UHIs) often appear in the summer when residential areas in the city have problems with ventilation, accumulating a large amount of heat generated by building materials under the influence of solar radiation. Therefore, in Table 2, the image recorded by Landsat 8 satellite on June 4, 2021, is selected to calculate the surface temperature and locate the urban heat island.

Table 3 shows the results of calculating the NDVI index and the surface temperature of the city of Moscow on June 4, 2021.

Table 3. Table of NDVI values and Land surface temperature

Index value NDVI		Land surface temperature (LST) value (℃)		
Min	**Max**	**Min**	**average**	**Max**
-0.975	0.960	13.93	24.81	43.43

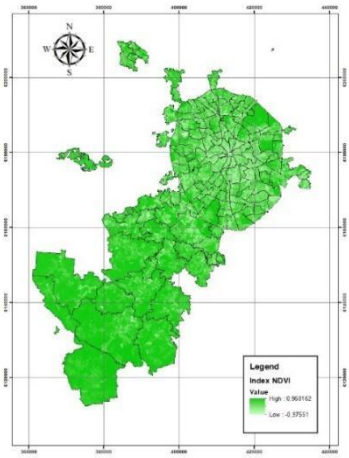

Fig. 2. The value of the NDVI index.

To determine surface emissivity, the NDVI index is an important parameter. Calculate the NDVI using formula (3) based on the surface reflectance spectral results. NDVI values range from -1 to 1. Plants usually have NDVI values greater than 0.2. In the NDVI field > 0.5, the area is considered to be covered with vegetation, and the electromagnetic radiation does not come from the ground. NDVI in the range [0; 0.2] corresponds to the bare land area, [-0.2; 1] represents the wet soil area, while the water raft has an NDVI value less than -0.2. On the NDVI index image, light-colored pixels represent areas of well-developed vegetation, while dark-colored pixels represent areas with no vegetation or low vegetation density. NDVI values above 0.7 are usually located in vegetated areas. The NDVI distribution shows that the maximum coverage is 0.96, and the lowest is -0.975 (Table 3). It can be seen that the high NDVI vegetation index is concentrated in the south of the city. Low coverage is concentrated in the city center. A spot where the NDVI is less than 0 occurs, such as lakes and rivers.

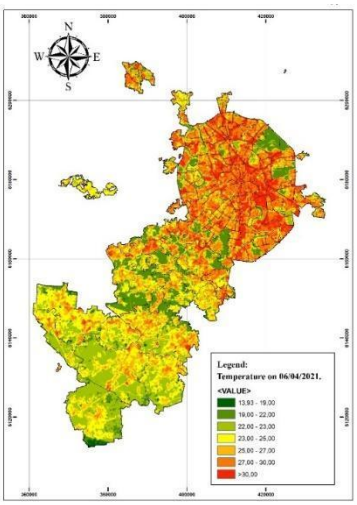

Fig. 3. Land surface temperature of Moscow on June 4, 2021.

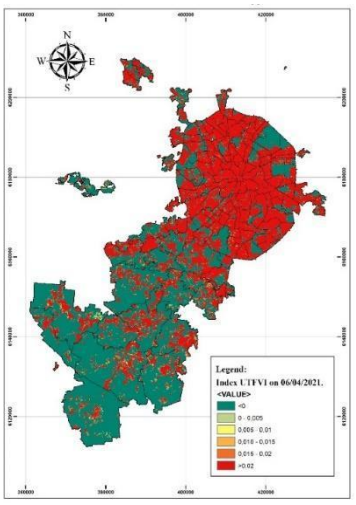

Fig. 4. Index UTFVI of Moscow on June 4, 2021.

The surface temperature of Moscow was determined using a Landsat 8 satellite image from June 4, 2021, revealing a maximum temperature of 43.43 ℃, a minimum temperature of 13.93 ℃, and an average temperature of 24.81 ℃. High temperatures were observed in densely populated downtown areas, while lower temperatures were observed in sparsely populated southern areas. The UTFVI index map was generated using the surface temperature data and formula (6) in Figure 4, with areas in red indicating strong urban heat islands (UTFVI values > 0.02), as shown in Table 1. Figure 4 provides a visual representation of the location of heat islands in Moscow.

4 Discussion

Due to its cloud-based geospatial processing power and vast collections of geospatial datasets such as Landsat and Modis, Google Earth Engine (GEE) has become increasingly popular in environmental and urban studies. However, despite its potential, ecological and urban modelers face three significant challenges when using GEE. Its current applications remain confined to basic mapping and underutilize its complete capabilities. Secondly, modelers must overcome technical complexities to develop image processing-based environmental models effectively. Lastly, many ecological and urban modelers are unaware of GEE's unprecedented geospatial processing capacity and an extensive collection of big geospatial datasets, resulting in the untapped potential of GEE to support ecological and urban modeling. Therefore, future studies aim to improve GEE's functions with custom and functional sets to model sustainable urban growth under the influence of Urban Heat Island (UHI) and Urban Population Increase (UPI).

5 Conclusion

This paper introduces the theoretical and experimental basis for determining the land surface temperature from the thermal infrared image of Landsat 8 Moscow region on June 4, 2021, by combining cloud computing and big data. The results indicate that the urban heat island phenomenon affects the area of Moscow. The research results also show that

using cloud computing to process large volumes of data has shortened the computation time and its availability in studying urban heat islands or changing climates. In addition, areas with many trees have a lower temperature, showing the potential of green strategies in mitigating the impact of urban heat islands.

6 Acknowledgments

We want to thank the planning department of the Moscow State University of Civil Engineering for supporting this study.

References

1. M. T. Le, T. A. Tuyet Cao, N. A. Quan Tran, S. I. Sadriavich, T. K. Phuong Nguyen, and T. K. Cuong Le. IOP Conference Series: Materials Science and Engineering, Institute of Physics Publishing, Nov. 2019.
2. Le Minh Tuan, I. Shukurov, and Nguyen Thi Mai. Stroitel stvo nauka i obrazovanie [Construction Science and Education], no. 3, Sep. 2019.
3. "Landsat 8 (L8) Data Users Handbook," 2019.
4. M. A. Wulder et al. Remote Sens Environ, vol. 185, pp. 271–283, 2016.
5. D. P. Roy et al. Remote Sens Environ, vol. 114, no. 1, pp. 35–49, 2010.
6. C. , L. X. , Z. M. , L. D. , & X. J. Cao. Environmental science, vol. 38, pp. 3987–3997, 2017.
7. Jeffrey Masek et al. IEEE geosience and remote sensing letters, vol. 3, no. 1, pp. 68–72, 2006.
8. K. Yamada, T. Hayasaka, and H. Iwabuchi. Scientific Online Letters on the Atmosphere, vol. 8, no. 1, pp. 94–97, 2012.
9. B. L. Zhuang et al. Atmos Environ, vol. 83, pp. 43–52, 2014.
10. X. Tie et al. Sci Rep, vol. 7, no. 1, Dec. 2017.
11. G. Mouri, S. Shinoda, and T. Oki. Environmental Modelling and Software, vol. 32, pp. 16–26, Jun. 2012.
12. T. Petäjä et al. Sci Rep, vol. 6, Jan. 2016.
13. Y. Wang et al. Atmos Environ, vol. 40, no. 16, pp. 2935–2952, May 2006.
14. M. C. Hansen and T. R. Loveland. Remote Sensing of Environment, vol. 122. pp. 66–74, Jul. 2012.
15. F. Chang et al. OSDI 2006 - 7th USENIX Symposium on Operating Systems Design and Implementation, pp. 205–218, 2006.
16. J. C. Corbett et al. ACM Transactions on Computer Systems, vol. 31, no. 3, pp. 1–22, 2013.
17. H. Gonzalez et al. Proceedings of the ACM SIGMOD International Conference on Management of Data, pp. 1061–1066, 2010.

18. A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes. Proceedings of the 10th European Conference on Computer Systems, EuroSys 2015, 2015.
19. X. She, L. Zhang, Y. Cen, T. Wu, C. Huang, and M. H. A. Baig. Remote Sens (Basel), vol. 7, no. 10, pp. 13485–13506, 2015.
20. M. I. Faridatul and B. Wu. ISPRS Int J Geoinf, vol. 7, no. 12, 2018.
21. L. Liu and Y. Zhang. Remote Sens (Basel), vol. 3, no. 7, pp. 1535–1552, 2011.